ROBOT DOCUMENTATION FOR WRO 2026 FUTURE ENGINEERS

Team Ouroboros

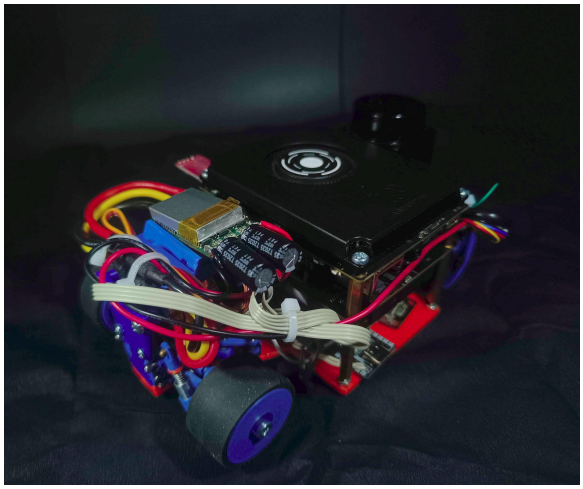
Authors:
Blazo Barovic
Lazar Milicevic
Milos Milicevic

Introduction **3**
Abstract **3**
Mobility & Mechanical Design **3**
Chassis Design Choices 3
Steering and Drive Mechanism Justification 4
Torque/speed reasoning 5
Mechanical Stability 5
Power & Sensor Architecture **5**
Power System 5
Processing Units 6
Sensor System 6
LiDAR 6
IMU 6
Camera 6
Encoder 6
Software Architecture & Obstacle Strategy **7**
Introduction 7
Explanation of algorithms 7
Localization Algorithm (ICP) 8
Signal handling 10
Signal Mapping 10
Waypoint Generation 10
Testing & Evaluation **11**
Mechanical Testing 11
Localization Accuracy 11
Algorithm Performance 11
Revisions & Improvements **11**
Overall Revisions 11
Tire Choice 12
Design Process 13
Conclusion **16**

Introduction

In this documentation, we describe the thought process, implementation, and design challenges encountered during the WRO 2026 Future Engineers challenge. The focus of this document is not only to present the final system, but also to clearly explain the reasoning behind key design decisions and the trade-offs considered during development.

In addition, we present the working principles of the robot, including its mechanical design and the implementation of the algorithms used in the system.



Abstract

During the development of the robot for this year's challenge, multiple design approaches were evaluated based on performance, complexity, and reliability. Due to these factors, the robot was designed as a four-wheel drive and four-wheel steering platform. This configuration was selected to improve traction, acceleration, and maneuverability, which are critical for navigating the competition environment.

The robot is equipped with passive suspension to ensure consistent contact of all four-wheels with the surface during operation. This improves stability and helps maintain reliable traction and sensor performance.

The robot utilizes multiple sensors to accurately determine its position and heading relative to the map. These sensors include a two-dimensional planar LiDAR sensor, a 9-DOF IMU, magnetic absolute position encoder and a 5-megapixel USB camera. Each sensor was selected for its specific advantages, and together they provide complementary data for robust perception and localization. The sensor data is processed and fused within the system.

For handling autonomous actions, the robot utilizes multiple algorithms corresponding to the challenges in this year's competition. We use three different algorithms:

- Pathfinding algorithm for the open challenge
- Pathfinding algorithm for the obstacle avoidance challenge
- All-purpose localization algorithm

During development, we evaluated the accuracy and execution speed of individual subsystems, as well as the system as a whole, in order to ensure reliable real-time performance within hardware constraints.

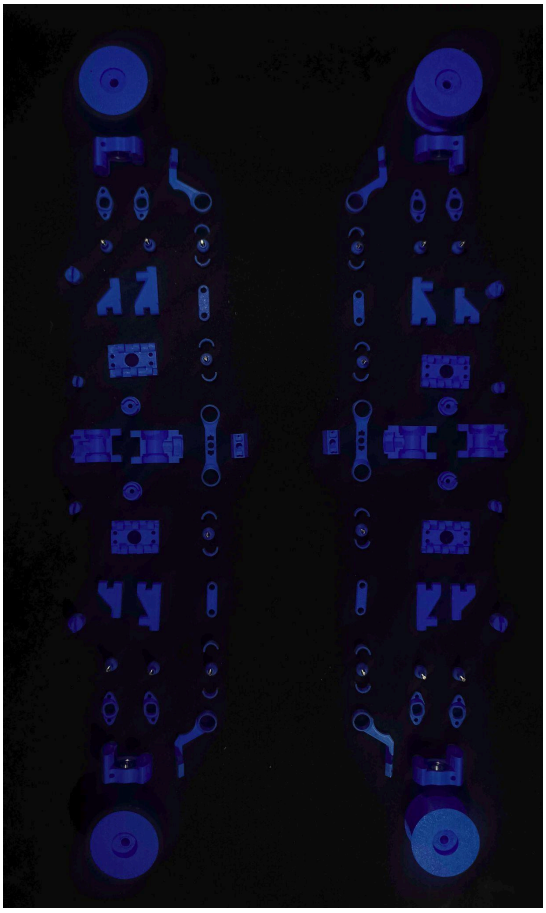
Mobility & Mechanical Design

Chassis Design Choices

During the development, the team has looked into multiple options for the chassis design based on the optimizations of the robot's speed, acceleration and turning radius. The decision was made to keep the size of the robot as small as possible. This reduces our weight and increases our clearance from signals and walls during the locomotion of the robot.

The robot is made using FDM (Fused Deposition Modeling) 3D printing technology. The plastic that is primarily used is PLA, but a select number of parts are made with ABS and TPU plastics. Other than plastic components, we use common mechanical parts such as bearings, screws, nuts, and heat inserts. We are also using off-the-shelf components such as: small form factor CVDs (Constant Velocity Drive) and open differentials, as well as metal gears. Some of the parts were manufactured out of aluminum using a lathe.

Robot mounting is primarily done by heat inserts and screws. Most of our screws mounting is done using M2 screws. Certain systems that are semi-decoupled from the robot base are attached with hinge joints and ball joints. These joints are made out of plastic using a 3D printer with a 0.25mm nozzle so that the feature size is as small as possible. This was a necessity because of the relatively small size of our robot (Figure 1.1).



Steering and Drive Mechanism Justification

For the optimization of the robot's speed and acceleration, we've decided to go with a four-wheel drive mechanism. This allows us to achieve higher acceleration by applying torque to all four-wheels, which increases our traction. Other factors that influence maximum speed and acceleration are choice of motor, gear ratio, tire diameter and material, excluding any aerodynamic effects.

Due to the additional challenge of parallel parking in the obstacle challenge, it was important to minimize turning radius. This also greatly benefits our ability to avoid signals and other collisions, significantly increasing our maneuverability. This is done by implementing a four-wheel steering

mechanism. This considerably decreases our turning radius compared to two wheel steering. However, due to the limitation of using a single steering motor, we are incapable of dynamically adjusting the steering ratio of the front and back wheels. This forces us to optimize for either low speed or high speed maneuvers. Due to the nature of the challenge, this makes it so that optimizing for low speed maneuvers is more beneficial, therefore the steering ratio between the front and back axle is 1:1. The steering is inspired by Ackermann steering geometry.

The drive utilizes two differentials, one on the front axle and one on the rear axle, enabling four-wheel drive. Due to the expected symmetrical load distribution between the front and rear axles, a third (central) differential is not required.

Instead, both axles are directly coupled, which simplifies the drivetrain and reduces overall backlash in the system.

Torque/speed reasoning

Torque and speed heavily depend on motor choice, but some other factors are gear ratio and wheel diameter, as well as the mechanical transmission efficiency. The motor we chose to go with is a 2830 1000-KV motor. This provides us with high speed and low torque. Torque directly from the motor would be insufficient, thus a gearbox is used. The gearbox ratio is 10:1, whilst the final drive ratio is 2:1, giving us a total ratio of 20:1. The wheel diameter is 34mm. This provides us with a balance between our torque and speed, allowing quick acceleration with a high maximum speed. The motor is more powerful than the use case requires, therefore it’s limited to a max power and speed more in line with our needs.

Mechanical Stability

Some systems don’t require high stability, whilst for others it's crucial for it to be stable with low backlash and higher rigidity. Based on these factors, we have designed our system for the needs of our robot, thus, parts such as the steering knuckle, wishbones, hinge joints, the drive mechanism and the steering system are made as rigidly as possible within the size constraint, whereas parts such as electronics mounting, the backplate are made with limited strength to save space as these parts do not require it.

Power & Sensor Architecture

The robot’s electrical and sensing architecture were designed to ensure reliable operation, efficient power distribution, and accurate data acquisition from all sensors.

Power System

The robot is powered using a 3S1P (3 Series, 1 Parallel) lithium-ion battery configuration. The battery was selected based on its energy density, discharge rate, and size constraints.

Power is distributed to different subsystems using Smart UPS Expansion Board for LattePanda IOTA (18650 Battery). Different voltage levels are required for various components:

- BLDC Motor: 12V, direct from battery
- Servo Motor: 12V, direct from battery

- Microcontroller STM32: 5V, from linear voltage regulator
- LiDAR: 5V, from USB
- BNO055 IMU: 3.3V, LattePanda IOTA breakout
- LattePanda 5MP UVC Camera: 5V, from breakout
- AS5600 Encoder: 3.3V, from STM32

Voltage regulation is handled using Smart UPS expansion board, ensuring stable operation and protecting sensitive electronics, as well as handling battery management functionality, such as overcharge protection, overdischarge protection and overcurrent protection.

Processing Units

The robot utilizes multiple processing units to handle different tasks:

- Main processor: Intel N150
- Microcontroller: STM32

The main processor is responsible for high-level tasks such as:

- Sensor fusion
- Localization
- Path planning
- Machine vision

The microcontroller is responsible for:

- Low-level motor control
- Servo control
- Real-time actuation

Sensor System

The robot uses a combination of sensors to achieve robust perception:

- LiDAR: Used for distance measurement and environment mapping
- IMU: Used for orientation and motion estimation
- Camera: Used for signal detection
- Encoder: Used to provide an approximate position estimate.

LiDAR

Model: STL-27L 360° DTOF LIDAR Sensor (UART, 25m)

- Range: 25m

The LiDAR provides point cloud data used for localization and obstacle detection, including environment mapping.

IMU

Model: BNO055

- Degrees of freedom: 9
- Data used: heading angle

The IMU provides high-frequency motion data used as an initial estimate in the localization algorithm.

Camera

Model: LattePanda 5MP UVC Camera

- Resolution: 5 MP

The camera is used for signal detection.

Encoder

Model: AS5600

The AS5600 is a high-precision, 12-bit contactless magnetic rotary position

sensor that provides absolute angle measurement over a 360-degree range. As we have attached it to the backside of the motor, where it measures rotation that can then be converted into distance traveled. The encoder was necessary because the accuracy of the robot was not ideal, therefore we needed to add a feedback loop.

Actuators

For the actuation and movement of the robot, only two actuators are used. One being the steering servo, whilst the other one is the drive motor.

The steering servo is a serial servo supplied with 12V. The servo has three wires. Such that they are: 12V, GND and data. The servo serial communication is a half duplex UART (Universal Asynchronous Receive Transmit) connection running at a baud rate of 1M. Its commands are sent by the STM32 microcontroller.

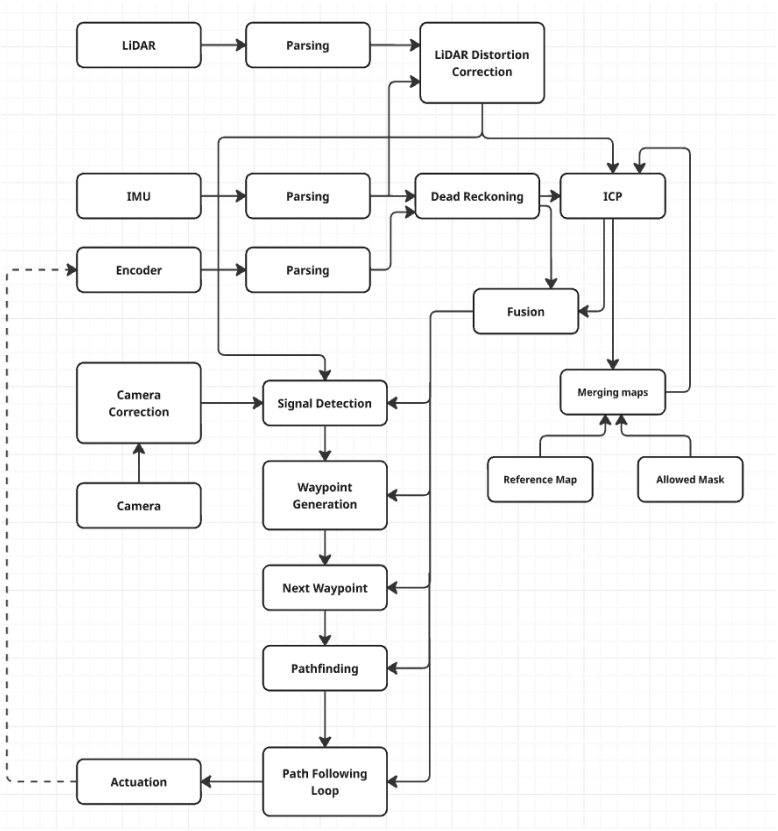
The drive motor is a 1000KV 11 pole BLDC (Brushless DC) motor. This kind of motor relies on electronic communication which is handled by an ESC (Electronic Speed Controller). For the purposes of this vehicle it was needed for the speed controller to support bidirectional commutation. This is not a default setting in many affordable ESC's used for such drone motors. For this reason, it was needed to flash a custom firmware that would enable such a feature. The firmware of choice was the popular SimonK. It's tailor made for such cheap ATmega based ESC's. The ESC is controlled by a PWM signal whose pulse width ranges from 1060-2060us. This PWM signal is generated by the STM32 microcontroller. It's supplied with 12V, same as the servo.

Software Architecture & Obstacle Strategy

Introduction

During the development of the robot software, it was the team's goal to minimize external dependencies such as libraries, executables, or SDKs. For this reason the only dependencies used are OpenCL, OpenCV and Arduino libraries (SCServo, Servo).

The system is built on top of the open source operating system Ubuntu. Due to the hardware limitations in such a small form factor it was important to choose a highly performant programming language that allowed for low access and was compatible with the aforementioned libraries. The choice boiled down to C or C++, and C++ was chosen due to previous experience with the language.



Explanation of algorithms

The main problem in robotics of this type boils down to 2 simple questions:

- *Where is the robot?*
- *Where to go next?*

To answer these questions many methods are used in the industry and research. The question of *Where is the robot?* is in essence a localization problem. Many robots on this scale use what is called **dead reckoning using odometry**. This is the process by which the robot's position is determined based on the previous robot position and a calculated or measured change in position from wheel encoders. But due to its iterative nature, any and all measurement errors will be added up, and they will tend to increase the error between our predicted position and real position. This is called drift. Previously on the WRO RoboMission 2025 our team developed such a system, but due to the previously mentioned drift as well as the significantly bigger track length of the WRO Future Engineers 2026 map that would amplify the total error further we decided to research other alternatives.

One method that is often mentioned in the literature is use of a local or global positioning system such as GPS or GNSS. But due to rule **11.6.** - **"**A vehicle must be autonomous and finish the "missions" by itself. Any radio communication, remote control, and wired control systems are not allowed while the vehicle is running. Teams in violation of this rule will be disqualified." that forbids external communication such methods are not useful for our use case.

Another method is SLAM (Simultaneous Localization and Mapping) using a combination of sensors. These sensors most often include **LIDARs** and **IMUs.** The SLAM system uses the LIDAR point cloud reading and matches it to a previously constructed map. This is most often done with the **ICP (Iterative Closest Point)** algorithm. The IMU provides an initial guess of the current transformation of the points. This guess is then used for the first iteration of ICP. This boosts convergence speeds and reduces possible errors, as long as the IMU readings are properly filtered. If the new readings are added to the map this will cause drift over time as small errors add up. This is often solved by a system often referred to as loop closure, but due to the algorithm's complexity and relatively small and known environment for this challenge we decided it would be better if we performed ICP using a pre-generated map. This map would be appended with the dynamic elements of the challenge (signals or randomized walls).

Prebuilt solutions for such a SLAM system exist within software packages such as ROS 2, but due to the previously stated self-imposed limitations on the software dependencies, we decided to implement our own even if its performance is worse than existing solutions.

Localization Algorithm (ICP)

The ICP algorithm is used to align LiDAR scans with a pre-generated map. The algorithm iteratively minimizes the distance between corresponding points. For each point, in a LiDAR scan, it finds the closest point from the reference map forming a point pair. This is done with a nearest neighbor search algorithm. The exact implementation is described later. After forming the pairs, a least squares solution for the transform (including translation and rotation) is solved. This

transform is then applied to the original scan and the process is repeated until the error converges or a maximum number of cycles is exceeded.

The nearest neighbor search (from now on **NNS**) is performed for all the points in the scan (around 2160) across all iterations (max. 100, average 20), in searches across a reference map containing approximately 10000 points. If a simple NNS algorithm that iterates for each point in the scan across the whole reference map to find the nearest point it quickly becomes computationally expensive. This approach would be unable to perform ICP in the 100ms between each LiDAR scan. A different implementation had to be used.

A different data structure for the reference map was the most effective solution. A spatially indexed data structure was chosen due to the relatively static nature of the reference map. Two options were considered:

1. K-D Tree
2. Quad tree

The main deciding factor was the simplicity of tree generation. Quad trees were easier to generate.

To further save on computation an approximate NNS algorithm was used. This involved only navigating down the quad tree once per point without backtracking. To accommodate this change, the reference map is rotated by 45° to decrease the chances of a line of points, aligning with the quad border.

This was still underperforming, so the next best optimization was multithreading. To take advantage of the onboard GPU for general purpose parallelized computation, an OpenCL kernel was used. As per recommendations in the OpenCL™ Developer Guide for Intel® Processor Graphics[1]. The biggest performance increase comes from proper and efficient memory usage. To aid in this we decided to decrease the overall size of the quad tree data structure. Each quad is only 8 bytes big such that it contains an **id** (4 bytes) that describes the exact position of the quad and its size. The remaining 4 bytes are used for storing the children offset in the array and their number.

The whole data structure is stored in an array of quads with the first element being the quad tree root. In addition to this the position and size of the root elements are passed in as 3 constant values (x, y, width).

To further optimize the runtime of the OpenCL kernel we only pass in the scan points once, and on subsequent iterations of the ICP we only update the transform that is applied to these points. It might seem counterintuitive to perform the point transformation twice, once on the CPU and once on the GPU, but the memory bandwidth reduction greatly outweighs any computational increase.

After implementing all these optimizations the execution of ICP for a single scan goes from **>1s** for the naive NNS algorithm to below **50ms**. The algorithm only uses one CPU thread and therefore allows for additional processes such as communication, control loops and related processes.

[1] https://www.intel.com/content/www/us/en/content-details/774834/opencl-developer-guide-for-intel-processor-graphics.html

Creating & Updating Reference Map

The reference map used in the ICP starts with already predefined points such as the outer walls and in case of obstacle challenge the parking spot. This reference map is dynamic, points are added to it as the robot explores its environment. The points that are added are first put through a mask. This mask blocks out the addition of any points on the drive surface, excluding signal positions. We have a secondary check before adding any points. These points must temporally agree on their positions in multiple previous readings before being added.

LiDAR Distortion Correction

To improve the accuracy and stability of the ICP algorithm, it is necessary to correct the LiDAR data in its uncorrected form. The points from LiDAR are distorted due to the robot rotating during the scan. We correct for this rotation with historical data from the IMU.

Dead Reckoning

Dead Reckoning is the process in which we calculate the changing position and rotation based on inertial measurements of the robot. These measurements are heading from the IMU and motor encoder.

Fusion

We are using the simplest sensor fusion implementation in case ICP fails to converge to a low cost value. We fall back on dead reckoning for position, until the ICP converges again.

Pathfinding Algorithm

The pathfinding algorithm is an algorithm that takes in the environment map, current position, system parameters, target position information and returns us a valid path to our end goal. The algorithm takes into account both position and rotation of the robot as well as the target position. The environment map is a point cloud. It consists of points that represent the walls and signals as well as points that represent virtual walls constraining our movement to the desired path. A valid path is any path that the robot can execute without colliding with signals or walls and that gets it to the target position.

To achieve this we are using the Hybrid A* algorithm tailored to a Reeds-Shepp vehicle. This vehicle is described with a minimal turning radius. It can only perform one of six actions: move forwards and backwards, steer left or right during those motions as shown on the figure (Figure 1.2).

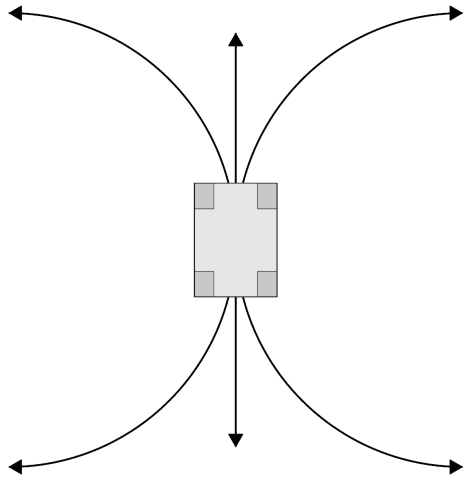


Figure 1.2.

Hybrid A* algorithm starts with a single node which is described by the vehicle's initial position and rotation. As this

the only node, it is used for generation of six additional nodes. These new nodes correspond to the positions and rotations of the robot when traveling along each of the six possible movements for a distance equaling the step size. Each of these nodes check if the robot would collide with the environment in that position, this is a simple radial distance check. If it does collide, the node is deleted. For the remaining nodes, a cost is calculated. This cost is equal to the length of the shortest Reeds-Shepp path. These new nodes are inserted into an array of possible future positions. For the next iteration, the starting node is picked from this array, such that it is the element with the least cost. This process is repeated until one of three conditions are met:

1. There are no more possible nodes, in which case the algorithm would return no path.
2. If the computation time is exceeded, it returns no path.
3. If a valid Reeds-Shepp path is found from the last explored node to the target position, such that the robot does not collide with the environment whilst it follows that path. Then we return the Hybrid A* path with appended Reeds-Shepp path.

To determine if the Reeds-Shepp path produces a collision with the environment, we use an approach similar to ray marching where we iteratively go along the path for a minimum distance from the environment and on each spot we check if that minimum distance is less than a collision threshold. If for any of these points that condition is true, the path results in a collision.

Starting Position

To find the initial position of the robot on the map, we used the previously described ICP algorithm and run a single iteration from each possible starting position and rotation and pick one with the lowest cost. Due to the symmetry of the map's external walls, only twelve unique starting positions for the open challenge exist and only two for the obstacle challenge.

Signal handling

To perform signal handling we have to know three things: the drive direction of the vehicle during a round, the color of the signals and their positions. We have to know all three of these to be able to determine the positions of waypoints in global scope.

Signal Mapping

To determine the position and color of the signals, we use data from the camera, LiDAR and the position returned by the ICP algorithm. The first step is to calibrate the camera. This is done using OpenCV's implementation of camera calibration. Next, we crop the image to the ROI (Region Of Interest) such that we exclude the ground and the area above the walls. We convert the RGB (Red, Green, Blue) to HSV (Hue, Saturation, Value). First we create the mask by checking all of the pixel values that match a value range of a color. In the mask, we find contours, iterate over them and delete any that are too small.

We redraw the image from the remaining contours.

We match the FOV of the camera to a region of the LiDAR's FOV. Afterward, we find the average distance and angle from the LiDAR to a detected signal. This distance and angle are converted to global coordinates. We store these positions in an array. If four separate calculations agree that the signal is found close to a possible position, we will add a signal to that position. We separately check for green and red.

Waypoint Generation

Waypoints are used in our pathfinding as target positions. These waypoints are crucial for our signal handling as they determine the factoring path chosen by the robot. When the robot is moving in a CCW direction and detects a green signal, it generates a waypoint that is located inwards a certain distance away from the signal and if it sees a red signal, it generates a point outwards. This is reversed if the direction is reversed. The signals also generate a waypoint if the signal is located close to the corner segment, it also generates a waypoint that is located on the middle of its straight segment. The signal has the same direction as the drive direction. To aid in the single detection, we have four fixed waypoints one on each corner segment rotated towards the next straight segment. The robot follows these points in an order of the drive direction, i.e. CW or CCW.

Path Following

If the path is followed strictly based on the Reeds-Shepp path, a lot of jitter can be observed. For this reason, it was needed to perform path smoothing and correct for any deviation from the path in real time. To do this, we implemented a Stanley controller. This controller looks at the closest point on the path and the current state of the robot to determine the needed steering of the vehicle to get back onto the path.

This implementation had significant oscillations due to the response delay of the system, mainly the limited rate of change of the steering servo. To fix this, we introduced a predictive element that predicts the future position and rotation of the robot and uses it to calculate the command for the servo. The prediction window is tuned such that by the time we reach the predicted point in time, servo has moved to the position commanded to it at the time of making the prediction.

In addition to this, if we are nearing the end of the planned path, we command the drive motor to decelerate.

Testing & Evaluation

During development, the system was tested to evaluate performance, accuracy, and reliability.

Mechanical Testing

- Minimum turning radius: 330mm
- Maximum speed: 0.7$\frac{m}{s}$
- Speed used: 0.5 $\frac{m}{s}$
- Acceleration: 0.25$\frac{m}{s^2}$

Algorithm Performance

- Average computation time for the ICP algorithm: 5-90ms
- Average computation time for the pathfinding algorithm: 0-20ms

Revisions & Improvements

Overall Revisions

During development, we had two major revisions: the first was purely conceptual design made fully parametric

and fully adjustable for the sake of validation out the suspension and steering geometry.

This design was large, bulky and had fundamental flaws for the design. The good part about the initial design was its parametric nature. That allowed us to quickly adapt it after the initial time invested into making it parametric. This revision was primarily used before we committed to any choice of hardware, such as bearings, tires, CVD's, differentials, gearbox, motor, etc.

The next major change was an overall reduction of size of all parts. Before any big changes, we aimed to evaluate the hardware we could use. We had a vast selection of parts to choose from on the bearing front, as well as on the drive train or drive base parts such as: CVD's, shafts and differentials.

Tire Choice

But our biggest problem at the time was the material for tires. We were unable to source two part silicone, thus we needed to find a more readily available solution. After evaluation our choices, we had two options:

1. Caulk silicone
2. Silicone bike grips

From our previous experiences, we knew that we could manufacture tires out of caulk silicone. These tires would have pretty good performance, but we still wanted to evaluate the other options we did not try in the past that, after some research, we thought the second option of silicone grips could have better or equal performance than caulk silicone.

However, it depends on the material used and the geometry of the grips. We decided our best course of action was to check in local bicycle shops their selections of handle grips. We found what we were searching for, and evaluated that it had slightly better traction and was significantly sturdier than a solution made with caulk silicone.

Because of the fixed size of the inner diameter of the silicone grips, that was the approximate size we had to go with for our rims. This size could be slightly bigger as the silicone is stretchable. At the end, we decided to have a 26mm outer diameter of our rims whilst the grips had an inner diameter of 21mm. This ensures that the grips will not slip on the rim even with no patterns on the surface of the rim. The inside diameter of the rim is 24.5mm. From this choice, the scale of our steering knuckle and all the connected mechanisms were derived.

In this revision, the scale of the entire mechanism was reduced by a factor of two. Most of the hardware, i.e. bearings and joint sizes were finalised and a basic untuned steering geometry was implemented. The third and final revision mostly dealt with tuning: steering, gear meshing, suspension mechanisms, as well as tolerances for printed 3D-printed parts.

The first implemented steering was simplified Ackermann steering geometry. Whilst the subsequent revisions dealt with adapting such geometry to work with suspension and within our size constraints.

The following figures show our steering geometry in different development stages.

Figure 1.

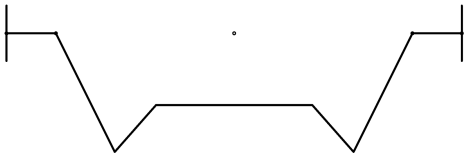


Figure 2.

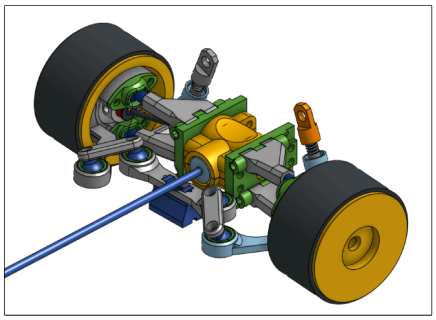


Figure 3.

Design Process

First version of the steering knuckle. This version was simplistic and meant as a proof of concept. It contained minimal features and did not satisfy our criteria, it was difficult to implement and hard to alter the CAD design (Figure 4.).

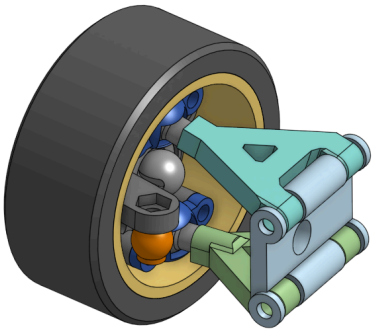


Figure 4.

Rework of the steering knuckle. Included caster angle and KPI in the initial base design. Did a fully parametric design for fast revisions. Finalized the design of the ball joints, connecting the steering knuckle to the wishbones (Figure 5.).

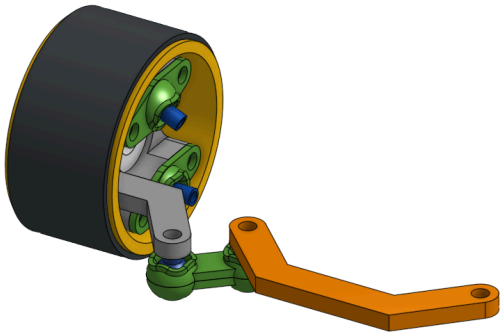


Figure 5.

Changed designed parameters, finalized size and determined tolerances used in printing of joints. Determined the usefulness of dry lubricants such as graphite, which did improve the performance of these joints (Figure 6.).

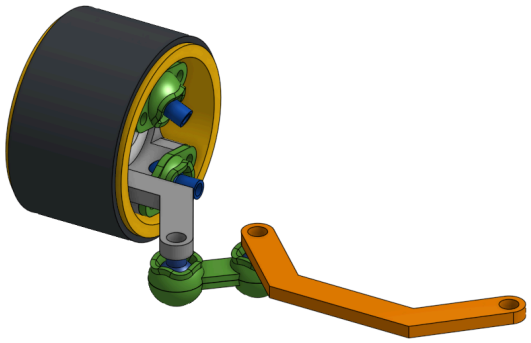


Figure 6.

Altered the ball joint design, used in the steering mechanism finalized its size and tolerances (Figure 7.).

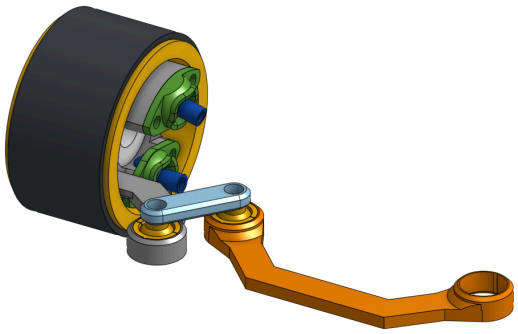


Figure 7.

Implementation of wishbones into the design, determining the hinge joints design and starting finalizing related dimensions (Figure 8.).

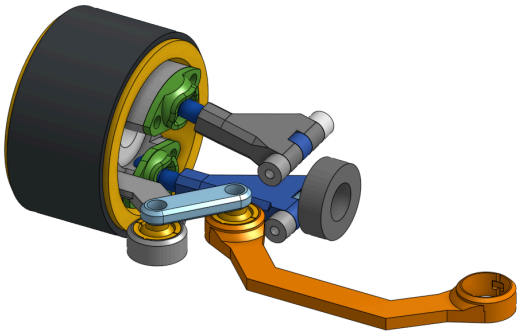


Figure 8.

Finalized the hinge joint and started work on revisions of both the steering geometry and differential housing (Figure 9.).

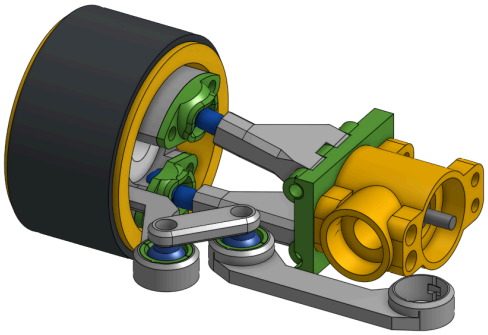


Figure 9.

Finished the design of steering geometry with slight tuning left to be done when the robot design is done (Figure 10.).

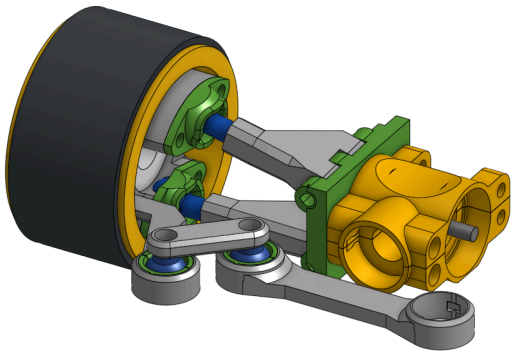


Figure 10.

Made the motor mount and started work on the actuation mechanism of the steering (Figure 11.).

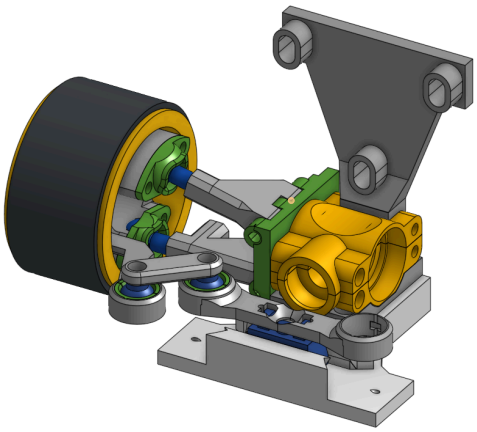


Figure 11.

Changed the motor from a 150ku BLDL to a 2830 1000kv BLDC motor with a metal gear box (Figure 12.).

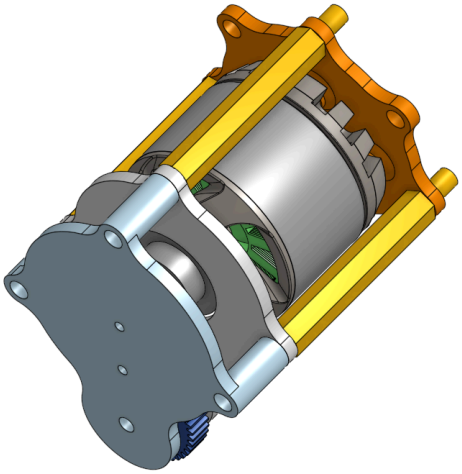


Figure 12.

Added camber angle to the design, improving tire contact with the driving surface during cornering and increasing lateral stability (Figure 13.).

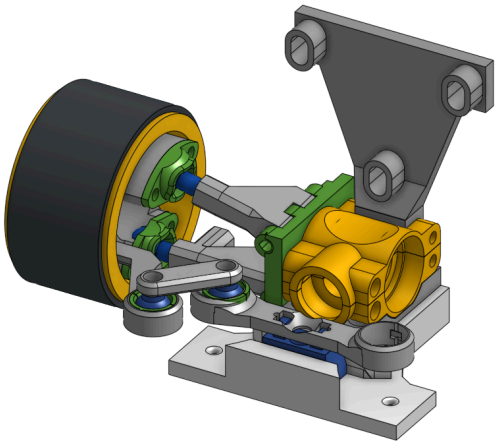


Figure 13.

Added passive suspension to the design to maintain consistent wheel contact with the ground and improve stability over uneven surfaces and during load variations (Figure 14.).

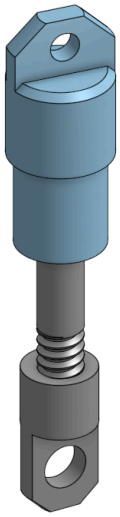


Figure 14.

Finalized steering actuation, changed from series string pulley design to a parallel string pulley design. Changed from a single strand nylon string to a twisted

fishing line made to not stretch or deform (Figure 15. & 16.).

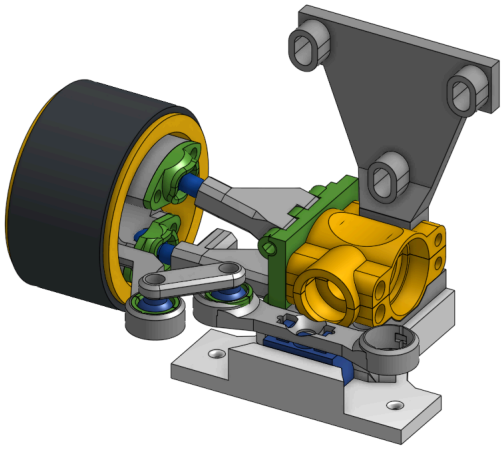


Figure 15.

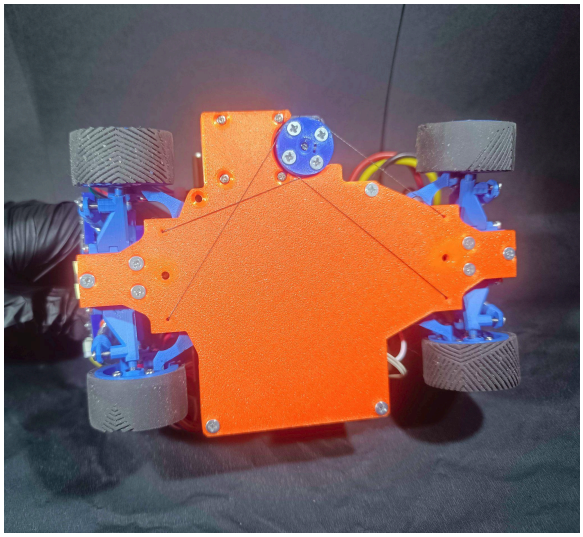


Figure 16.

The back board on which all the components are connected, changed tolerances and string channel sizes as well as added alignment holes used when tightening the steering actuation (Figure 17.).

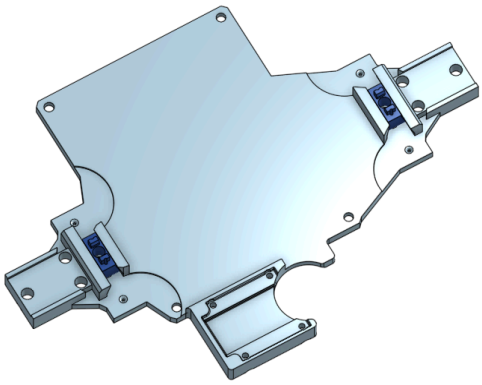


Figure 17.

Conclusion

The developed robot represents a carefully balanced integration of mechanical design, sensor architecture, and software systems, with a strong emphasis on reliability, efficiency, and real-time performance.

Throughout the development process, each subsystem was iteratively refined and evaluated to ensure that it meets the constraints and challenges defined by the WRO 2026 Future Engineers competition.

The combination of a four-wheel drive and four-wheel steering platform provides high maneuverability and precise control, which is essential for navigating complex environments. The use of a LiDAR-based localization system, supported by IMU data and optimized through a custom implementation of the ICP algorithm, enables accurate and stable position estimation without reliance on external infrastructure.

In addition, the integration of path planning and obstacle handling algorithms

allows the robot to adapt to both static and dynamic elements of the environment, ensuring safe and efficient navigation. Special attention was given to computational performance, resulting in a system capable of operating within strict real-time constraints.

Although further improvements can be made in certain areas, the current implementation demonstrates a robust and optimized solution. Overall, the system reflects a comprehensive engineering approach, combining theoretical understanding with practical design decisions to successfully address the requirements of the competition.